

CSE-4255: Introduction to Data Mining and Warehousing

Lab Experiment 1: OLAP Analysis on WideWorldImportersDW

Submitted by:

Tazkia Malik

Roll: 07

Submission Date: 5 October 2026



**Department of Computer Science and Engineering
University of Dhaka**

Contents

1	Introduction	3
2	Dataset Collection	3
3	Database Environment	3
4	Schema Design and Analysis	3
4.1	Schema Overview	3
4.2	Overall Fact-Constellation Diagram	4
4.2.1	Sales Star-Schema Example	4
4.3	Fact and Dimension Tables	7
4.4	Schema Classification	8
4.5	Measures and Measure Classification	8
4.6	Concept Hierarchies	9
4.7	Relationship Discussion	10
4.8	Schema Design Requirements Checklist	10
5	OLAP Operations	10
5.1	CUBE Operation	10
5.1.1	CUBE Query 1: Sales by Calendar Year and Country	10
5.1.2	CUBE Query 2: Sales by Customer Category and Calendar Year	12
5.1.3	CUBE Query 3: Orders by Stock Item Brand and Country	12
5.1.4	CUBE Query 4: Purchases by Supplier and Calendar Year	13
5.1.5	CUBE Query 5: Transactions by Type, Payment Method, and Calendar Year	13
5.1.6	CUBE Experiment Summary	14
5.2	ROLLUP Operation	14
5.2.1	ROLLUP Query 1: Geographic Sales Hierarchy	14
5.2.2	ROLLUP Query 2: Order Calendar Hierarchy	16
5.2.3	ROLLUP Query 3: Product Hierarchy in Orders	16
5.2.4	ROLLUP Query 4: Customer Buying Group Hierarchy	16
5.2.5	ROLLUP Query 5: Movement Transaction Type to Stock Item	16
5.3	Drill-Down Operation	17
5.3.1	Drill-Down Experiment 1: Sales Year to Month to Date	17
5.3.2	Drill-Down Experiment 2: Country to State to City	17
5.3.3	Drill-Down Experiment 3: Customer Category to Customer	19
5.3.4	Drill-Down Experiment 4: Product Brand to Stock Item	19
5.4	Slice Operation	19
5.4.1	Slice Query 1: Calendar Year 2015	19
5.4.2	Slice Query 2: Country United States	20
5.4.3	Slice Query 3: Supplier Fabrikam, Inc.	20
5.4.4	Slice Query 4: Customer Invoice Transactions	20
5.4.5	Slice Query 5: Stock Issue Movements	20
5.5	Dice Operation	20
5.5.1	Dice Query 1: 2015 United States Sales	21
5.5.2	Dice Query 2: Novelty Shop Profit in 2014–2015	21
5.5.3	Dice Query 3: 2015 Fabrikam and Litware Purchases	21

5.5.4	Dice Query 4: United States Northwind Orders	21
5.5.5	Dice Query 5: 2015 EFT Supplier Invoices	21
5.6	Pivot Operation	23
5.6.1	Pivot Query 1: Sales by Country Across Years	23
5.6.2	Pivot Query 2: Profit by Customer Category Across Years	23
5.6.3	Pivot Query 3: Ordered Quantity by Brand Across Years	24
5.6.4	Pivot Query 4: Purchases by Supplier Across Years	24
5.6.5	Pivot Query 5: Transactions by Type Across Payment Methods	24
5.7	Additional OLAP Operations	24
5.7.1	GROUPING SETS	24
5.7.2	Top-N and Ranking Analysis	25
6	Comparison of OLAP Operations	26
7	Observations and Findings	26
8	Conclusion	26
9	References	26

1 Introduction

This report presents Lab Experiment 1 for CSE-4255: Introduction to Data Mining and Warehousing. The experiment focuses on Online Analytical Processing (OLAP) analysis over the Microsoft WideWorldImportersDW data warehouse. The analysis will use dimensional-model concepts, including fact tables, dimension tables, measures, hierarchies, and OLAP operations.

2 Dataset Collection

The dataset used in this experiment is Microsoft WideWorldImportersDW, a moderate-sized official Microsoft data warehouse sample obtained from the authenticated Microsoft Learn catalog. It models business activities such as sales, purchasing, inventory movement, and stock holding. The warehouse contains multiple fact and dimension tables, making it suitable for studying dimensional modeling and performing OLAP analysis.

3 Database Environment

The WideWorldImportersDW database is restored locally in SQL Server 2022 running in Docker on macOS. The SQL Server container is named `sqlserver`, and the database is named `WideWorldImportersDW`. The database tool connection uses server `localhost` with the `sa` user. Queries for the experiment will be executed inside the running Docker container using the SQL Server command-line client.



```
DM git:(main) x docker exec sqlserver /opt/mssql-tools18/bin/sqlcmd \
-S localhost \
-U sa \
-P 'StrongPass!123' \
-C \
-d WideWorldImportersDW \
-Q "SELECT DB_NAME() AS DatabaseName;"
DatabaseName

-----
WideWorldImportersDW

(1 rows affected)
```

Figure 1: SQL Server 2022 Docker environment used to run the WideWorldImportersDW database locally.

4 Schema Design and Analysis

4.1 Schema Overview

The restored WideWorldImportersDW database contains eight dimension tables and six fact tables. The dimensions describe dates, locations, customers, employees, payment methods, stock items, suppliers, and transaction types. The facts record movements, orders, purchases, sales, stock holdings, and financial transactions. The inspected tables range from 6 rows in `Dimension.Payment Method` to 236,667 rows in `Fact.Movement`.

Their dimensional organization and declared foreign-key constraints are designed for analytical and data-warehouse workloads.

The schema inspection was performed using `INFORMATION_SCHEMA.COLUMNS`, `sys.tables`, `sys.columns`, `sys.indexes`, `sys.foreign_keys`, and `sys.foreign_key_columns`. Therefore, the keys, attributes, and links documented below are based on the restored database rather than on assumed names.

4.2 Overall Fact-Constellation Diagram

Figure 2 presents the complete verified fact constellation. All six fact tables and all eight dimensions are included, and every line represents a verified fact-to-dimension relationship. Shared dimensions are conformed across business processes; role-playing relationships such as multiple date, customer, and employee roles use the same dimension rather than duplicated boxes. The complete attributes are documented separately in the verified attribute catalog.

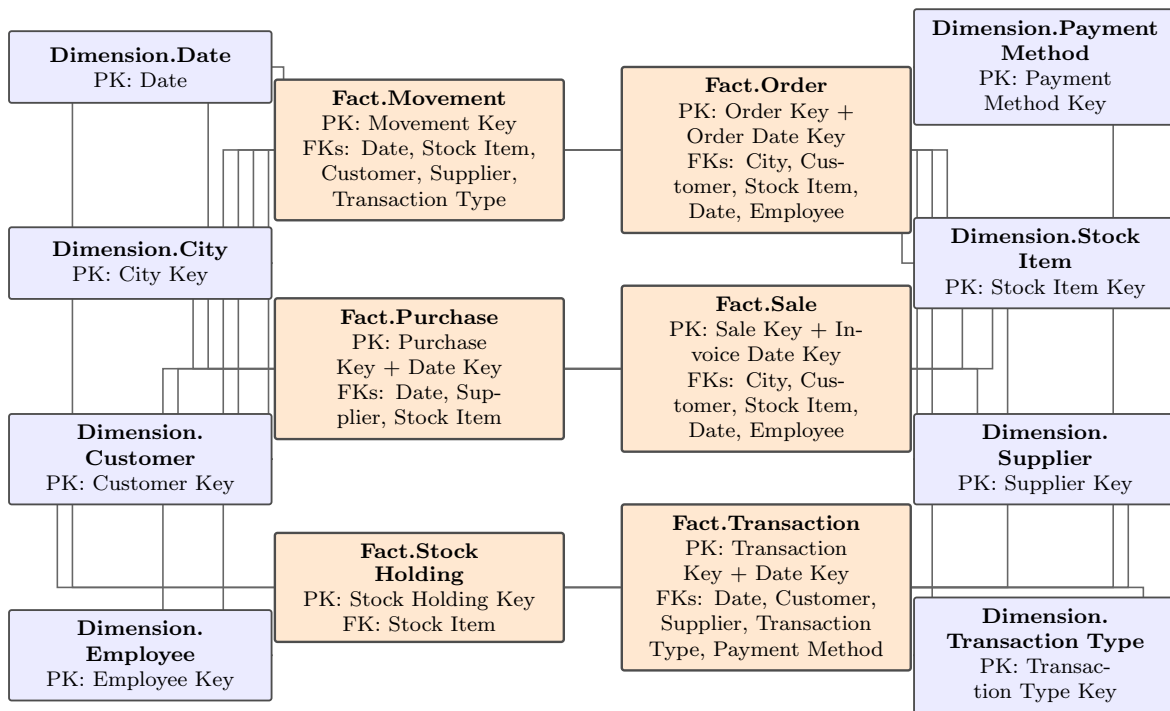


Figure 2: Overall fact-constellation schema of WideWorldImportersDW showing all six fact tables and eight dimension tables. Shared dimensions connect multiple business processes; role-playing relationships such as multiple date and customer roles are represented by the same conformed dimension.

4.2.1 Sales Star-Schema Example

The following figure is retained as a detailed, simplified sales subject-area example. It shows the measurable sales event and its descriptive dimensions, including the role-playing date, customer, and employee relationships.

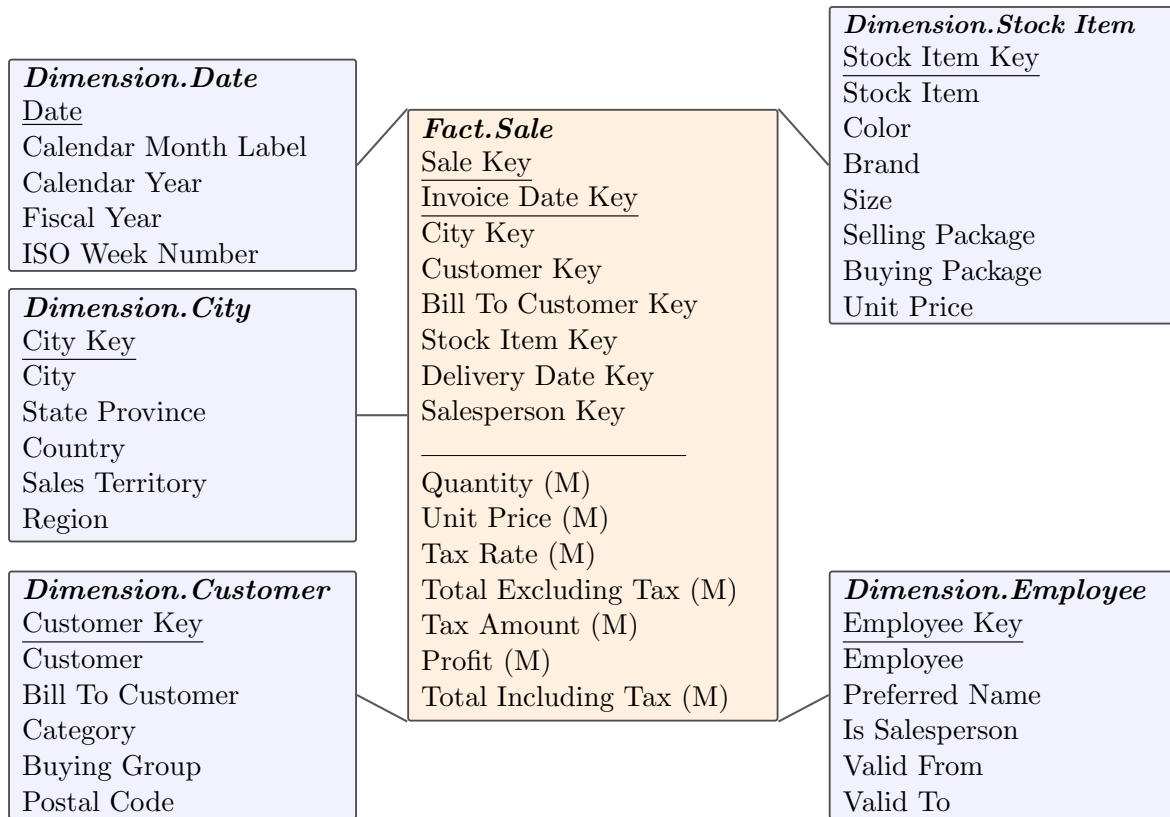


Figure 3: Simplified sales subject-area star schema for Fact.Sale and its connected dimensions. PK fields are underlined; FK fields are the related key fields; M denotes a stored measure.

The complete attribute inventory used to construct the diagram and analysis is given in Table 1. In the catalog, **PK** marks a primary-key column, **FK** marks a column participating in a verified foreign key, and **M** marks a stored numeric business measure.

Table 1: Complete verified attribute catalog

Table	Key	Attributes from metadata
Dimension.City	PK: City Key	City Key; WWI City ID; City; State Province; Country; Continent; Sales Territory; Region; Subregion; Location; Latest Recorded Population; Valid From; Valid To; Lineage Key
Dimension.Customer	PK: Customer Key	Customer Key; WWI Customer ID; Customer; Bill To Customer; Category; Buying Group; Primary Contact; Postal Code; Valid From; Valid To; Lineage Key
Dimension.Date	PK: Date	Date; Day Number; Day; Month; Short Month; Calendar Month Number; Calendar Month Label; Calendar Year; Calendar Year Label; Fiscal Month Number; Fiscal Month Label; Fiscal Year; Fiscal Year Label; ISO Week Number

Table	Key	Attributes from metadata
Dimension.Employee	PK: Employee Key	Employee Key; WWI Employee ID; Employee; Preferred Name; Is Salesperson; Photo; Valid From; Valid To; Lineage Key
Dimension.Payment Method	PK: Payment Method Key	Payment Method Key; WWI Payment Method ID; Payment Method; Valid From; Valid To; Lineage Key
Dimension.Stock Item	PK: Stock Item Key	Stock Item Key; WWI Stock Item ID; Stock Item; Color; Selling Package; Buying Package; Brand; Size; Lead Time Days; Quantity Per Outer; Is Chiller Stock; Barcode; Tax Rate; Unit Price; Recommended Retail Price; Typical Weight Per Unit; Photo; Valid From; Valid To; Lineage Key
Dimension.Supplier	PK: Supplier Key	Supplier Key; WWI Supplier ID; Supplier; Category; Primary Contact; Supplier Reference; Payment Days; Postal Code; Valid From; Valid To; Lineage Key
Dimension.Transaction Type	PK: Transaction Type Key	Transaction Type Key; WWI Transaction Type ID; Transaction Type; Valid From; Valid To; Lineage Key
Fact.Movement	PK: Movement Key; FKs: 5	Movement Key; Date Key (FK); Stock Item Key (FK); Customer Key (FK); Supplier Key (FK); Transaction Type Key (FK); WWI Stock Item Transaction ID; WWI Invoice ID; WWI Purchase Order ID; Quantity (M); Lineage Key
Fact.Order	PK: Order Key + Order Date Key; FKs: 7	Order Key; City Key (FK); Customer Key (FK); Stock Item Key (FK); Order Date Key (FK); Picked Date Key (FK); Salesperson Key (FK); Picker Key (FK); WWI Order ID; WWI Backorder ID; Description; Package; Quantity (M); Unit Price (M); Tax Rate (M); Total Excluding Tax (M); Tax Amount (M); Total Including Tax (M); Lineage Key
Fact.Purchase	PK: Purchase Key + Date Key; FKs: 3	Purchase Key; Date Key (FK); Supplier Key (FK); Stock Item Key (FK); WWI Purchase Order ID; Ordered Outers (M); Ordered Quantity (M); Received Outers (M); Package; Is Order Finalized; Lineage Key

Table	Key	Attributes from metadata
Fact.Sale	PK: Sale Key + Invoice Date Key; FKs: 7	Sale Key; City Key (FK); Customer Key (FK); Bill To Customer Key (FK); Stock Item Key (FK); Invoice Date Key (FK); Delivery Date Key (FK); Salesperson Key (FK); WWI Invoice ID; Description; Package; Quantity (M); Unit Price (M); Tax Rate (M); Total Excluding Tax (M); Tax Amount (M); Profit (M); Total Including Tax (M); Total Dry Items (M); Total Chiller Items (M); Lineage Key
Fact.Stock Holding	PK: Stock Holding Key; FKs: 1	Stock Holding Key; Stock Item Key (FK); Quantity On Hand (M); Bin Location; Last Stocktake Quantity (M); Last Cost Price (M); Reorder Level (M); Target Stock Level (M); Lineage Key
Fact. Transaction	PK: Transaction Key + Date Key; FKs: 6	Transaction Key; Date Key (FK); Customer Key (FK); Bill To Customer Key (FK); Supplier Key (FK); Transaction Type Key (FK); Payment Method Key (FK); WWI Customer Transaction ID; WWI Supplier Transaction ID; WWI Invoice ID; WWI Purchase Order ID; Supplier Invoice Number; Total Excluding Tax (M); Tax Amount (M); Total Including Tax (M); Outstanding Balance (M); Is Finalized; Lineage Key

4.3 Fact and Dimension Tables

Table 2: Dimension tables verified in the restored database

Dimension Table	Primary Key	Important Attributes	Rows
City	City Key	City, State Province, Country, Con- tinent, Region, Subregion	116295
Customer	Customer Key	Customer, Bill To Customer, Cate- gory, Buying Group, Postal Code	403
Date	Date	Day, Month, Calendar Year, Fiscal Year, ISO Week Number	1461
Employee	Employee Key	Employee, Preferred Name, Is Sales- person	213
Payment Method	Payment Method Key	Payment Method	6
Stock Item	Stock Item Key	Stock Item, Brand, Color, Size, packages, prices, Tax Rate	672
Supplier	Supplier Key	Supplier, Category, Primary Con- tact, Payment Days, Postal Code	28
Transaction Type	Transaction Type Key	Transaction Type	15

Table 3: Fact tables and verified connected dimensions

Fact Table	Primary Key	Connected Dimensions	Rows
Movement	Movement Key	Date, Stock Item, Customer, Supplier, Transaction Type	236667
Order	Order Key + Order Date Key	City, Customer, Stock Item, Date (order/picked), Employee (salesperson-/picker)	231412
Purchase	Purchase Key + Date Key	Date, Supplier, Stock Item	8367
Sale	Sale Key + Invoice Date Key	City, Customer (customer/bill-to), Stock Item, Date (invoice/delivery), Employee (salesperson)	228265
Stock Holding	Stock Holding Key	Stock Item	227
Transaction	Transaction Key + Date Key	Date, Customer (customer/bill-to), Supplier, Transaction Type, Payment Method	99585

4.4 Schema Classification

The complete warehouse is best classified as a **fact constellation (galaxy) schema** composed of several related star-like subject areas. The classification follows from the verified metadata: six fact tables share conformed dimensions such as Date, Customer, Stock Item, and Supplier, while other facts connect to specialized dimensions such as Payment Method or Transaction Type. An individual process, such as sales or purchasing, can be viewed as a star schema around its fact table. At the warehouse level, however, the presence of multiple interconnected fact tables makes fact constellation the more accurate overall description. The dimensions are not connected to one another by declared foreign keys in the inspected model, so there is no basis for calling the complete design a snowflake schema.

4.5 Measures and Measure Classification

The numeric-column inspection separated identifiers, keys, flags, and lineage values from business measures. A stored base measure is not intrinsically distributive, algebraic, or holistic; the classification depends on the aggregate applied to it. The table below records common analytical use. Sums and counts are distributive, while averages such as average unit price are algebraic because they can be computed from distributive summaries (sum and count). No holistic measure, such as a median, is stored or required by the inspected schema.

Table 4: Stored business measures and typical OLAP classifications

Fact	Stored measure	Meaning	Typical aggregation	Class.
Movement	Quantity	Moved quantity	SUM; COUNT	Distributive
Order	Quantity	Ordered quantity	SUM; COUNT	Distributive
Order	Unit Price	Price per order line	AVG; SUM when line totals are intended	Algebraic for AVG; distributive for SUM
Order	Tax Rate	Tax rate on order line	AVG; MIN/MAX	Algebraic for AVG; distributive for MIN/MAX

Fact	Stored measure	Meaning	Typical aggregation	Class.
Order	Total Excluding Tax	Order value before tax	SUM	Distributive
Order	Tax Amount	Tax charged on order	SUM	Distributive
Order	Total Including Tax	Order value including tax	SUM	Distributive
Purchase	Ordered Outers	Ordered package count	SUM; COUNT	Distributive
Purchase	Ordered Quantity	Ordered item quantity	SUM; COUNT	Distributive
Purchase	Received Outers	Received package count	SUM; COUNT	Distributive
Sale	Quantity	Sold quantity	SUM; COUNT	Distributive
Sale	Unit Price	Price per sale line	AVG; SUM when line totals are intended	Algebraic for AVG; distributive for SUM
Sale	Tax Rate	Tax rate on sale line	AVG; MIN/MAX	Algebraic for AVG; distributive for MIN/MAX
Sale	Total Excluding Tax	Sale value before tax	SUM	Distributive
Sale	Tax Amount	Tax charged on sale	SUM	Distributive
Sale	Profit	Profit on sale line	SUM	Distributive
Sale	Total Including Tax	Sale value including tax	SUM	Distributive
Sale	Total Dry Items	Dry-item quantity in sale	SUM; COUNT	Distributive
Sale	Total Chiller Items	Chiller-item quantity in sale	SUM; COUNT	Distributive
Stock Holding	Quantity On Hand	Current held quantity	SUM for inventory snapshot; AVG over snapshots	Distributive for SUM; algebraic for AVG
Stock Holding	Last Stocktake Quantity	Quantity at last stocktake	SUM; AVG	Distributive for SUM; algebraic for AVG
Stock Holding	Last Cost Price	Cost price at last stocktake	AVG; MIN/MAX	Algebraic for AVG; distributive for MIN/MAX
Stock Holding	Reorder Level	Reorder threshold	AVG; MIN/MAX	Algebraic for AVG; distributive for MIN/MAX
Stock Holding	Target Stock Level	Target inventory threshold	AVG; MIN/MAX	Algebraic for AVG; distributive for MIN/MAX
Transaction	Total Excluding Tax	Transaction value before tax	SUM	Distributive
Transaction	Tax Amount	Tax charged on transaction	SUM	Distributive
Transaction	Total Including Tax	Transaction value including tax	SUM	Distributive
Transaction	Outstanding Balance	Unpaid balance	SUM; AVG	Distributive for SUM; algebraic for AVG

4.6 Concept Hierarchies

The following hierarchies use only levels represented by actual dimension columns. Some dimensions contain descriptive attributes but do not expose a declared multi-level hierarchy; those attributes are therefore not presented as invented hierarchies.

Table 5: Concept hierarchies supported by inspected dimension columns

Dimension	Hierarchy	OLAP use
Date	Calendar Year → Calendar Month Label → Date	Roll-up and drill-down by calendar period
Date	Fiscal Year → Fiscal Month Label → Date	Fiscal-period reporting
Date	ISO Week Number → Date	Weekly analysis
City	Continent → Country → State Province → City	Geographic roll-up and drill-down
Customer	Category → Customer; Buying Group → Customer	Customer-segment analysis
Employee	Is Salesperson → Employee	Salesperson versus non-salesperson grouping
Supplier	Category → Supplier	Supplier-category analysis
Stock Item	Brand → Stock Item	Brand-level product analysis

4.7 Relationship Discussion

The facts connect to dimensions through the 29 foreign-key relationships declared in the database. Date is a shared conformed dimension across all six fact processes. Customer and Stock Item are shared by several operational processes, while Supplier is shared by Movement, Purchase, and Transaction. City and Employee support order and sale analysis, Transaction Type supports Movement and Transaction, and Payment Method supports Transaction. The repeated role-specific links preserve different business meanings without duplicating the dimension tables. Conformed dimensions make it possible to compare measures across processes using common calendar, customer, product, supplier, location, or employee perspectives.

4.8 Schema Design Requirements Checklist

Table 6 records the completion status of each required schema-design task.

Table 6: Schema design task checklist

Item	Requirement	Status
2.1	Overall full schema relationship diagram included with all six facts and eight dimensions; complete attributes documented separately in the verified attribute catalog	Complete
2.2	Star/snowflake/fact-constellation label with justification based on verified foreign keys	Complete
2.3	Fact and dimension tables identified with primary keys, connected dimensions, and row counts	Complete
2.4	Every stored business measure listed and classified by its typical aggregate as distributive, algebraic, or holistic	Complete
2.5	Concept hierarchies listed using only actual Date, City, Customer, Employee, Supplier, and Stock Item columns	Complete

5 OLAP Operations

5.1 CUBE Operation

The SQL Server GROUP BY CUBE operation computes aggregations for all combinations of the selected dimensions. It therefore returns detailed rows, subtotals for each dimension, and a grand-total row. In the grouped dimension columns, NULL values can indicate a subtotal or grand total rather than a missing business value. The queries below replace those generated NULL values with explicit labels such as ALL YEARS and ALL COUNTRIES.

The queries use the verified surrogate-key joins documented in the schema section. The GROUPING() function identifies whether a column was aggregated by CUBE; consequently, it distinguishes a CUBE subtotal from an actual nullable dimension value. GROUPING_ID() is returned as a compact indicator of which dimensions are aggregated: zero denotes a detailed row, while larger values identify subtotal or grand-total combinations.

5.1.1 CUBE Query 1: Sales by Calendar Year and Country

Objective. Compare sold units and profit by invoice calendar year and country, including yearly, country, and overall totals.

SQL Query.

```
SELECT CASE WHEN GROUPING(d.[Calendar Year]) = 1 THEN 'ALL YEARS'
          ELSE CAST(d.[Calendar Year] AS varchar(10)) END AS [Calendar Year],
       CASE WHEN GROUPING(c.[Country]) = 1 THEN 'ALL COUNTRIES'
          ELSE c.[Country] END AS [Country],
       SUM(s.[Quantity]) AS [Units Sold], SUM(s.[Profit]) AS [Profit],
       GROUPING_ID(d.[Calendar Year], c.[Country]) AS [Grouping ID]
FROM [Fact].[Sale] s
JOIN [Dimension].[Date] d ON s.[Invoice Date Key] = d.[Date]
JOIN [Dimension].[City] c ON s.[City Key] = c.[City Key]
GROUP BY CUBE(d.[Calendar Year], c.[Country])
ORDER BY GROUPING_ID(d.[Calendar Year], c.[Country]),
         d.[Calendar Year], c.[Country];
```

Result. The query returned 10 rows. The complete output contained only the United States country member, so the table below shows the actual complete result.

Table 7: Actual output of CUBE Query 1 (10 rows)

Year	Country	Units Sold	Profit	GID
2013	United States	2,401,657	22,768,352.25	0
2014	United States	2,567,401	24,828,462.45	0
2015	United States	2,740,266	26,957,600.65	0
2016	United States	1,241,304	11,174,765.55	0
2013	ALL COUNTRIES	2,401,657	22,768,352.25	1
2014	ALL COUNTRIES	2,567,401	24,828,462.45	1
2015	ALL COUNTRIES	2,740,266	26,957,600.65	1
2016	ALL COUNTRIES	1,241,304	11,174,765.55	1
ALL YEARS	United States	8,950,628	85,729,180.90	2
ALL YEARS	ALL COUNTRIES	8,950,628	85,729,180.90	3

Interpretation. Rows with GID 0 are year-country detail rows. GID 1 removes the country dimension and gives a yearly subtotal, while GID 2 gives the all-years country subtotal. GID 3 is the grand total. The result shows total profit of 85,729,180.90 across 8,950,628 sold units.

```

DM git:(main) x docker exec sqlserver /opt/mssql-tools18/bin/sqlcmd \
-S localhost \
-U sa \
-P 'StrongPass!123' \
-C \
-d WideWorldImportersDW \
-s "," \
-W \
-Q "
SELECT
    CASE WHEN GROUPING(d.[Calendar Year]) = 1
        THEN 'ALL YEARS'
        ELSE CAST(d.[Calendar Year] AS varchar(10))
    END AS [Calendar Year],
    CASE WHEN GROUPING(c.[Country]) = 1
        THEN 'ALL COUNTRIES'
        ELSE c.[Country]
    END AS [Country],
    SUM(s.[Quantity]) AS [Units Sold],
    CAST(SUM(s.[Profit]) AS decimal(18,2)) AS [Profit],
    GROUPING_ID(d.[Calendar Year], c.[Country]) AS [GID]
FROM [Fact].[Sale] s
JOIN [Dimension].[Date] d
    ON s.[Invoice Date Key] = d.[Date]
JOIN [Dimension].[City] c
    ON s.[City Key] = c.[City Key]
GROUP BY CUBE(d.[Calendar Year], c.[Country])
ORDER BY
    GROUPING_ID(d.[Calendar Year], c.[Country]),
    d.[Calendar Year],
    c.[Country];
"
Calendar Year,Country,Units Sold,Profit,GID
-----,-----,-----,-----,-----
2013,United States,2401657,22768352.25,0
2014,United States,2567401,24828462.45,0
2015,United States,2740266,26957600.65,0
2016,United States,1241304,11174765.55,0
2013,ALL COUNTRIES,2401657,22768352.25,1
2014,ALL COUNTRIES,2567401,24828462.45,1
2015,ALL COUNTRIES,2740266,26957600.65,1
2016,ALL COUNTRIES,1241304,11174765.55,1
ALL YEARS,United States,8950628,85729180.90,2
ALL YEARS,ALL COUNTRIES,8950628,85729180.90,3

(10 rows affected)
DM git:(main) x

```

Figure 4: Execution of a CUBE query showing detailed rows, subtotals, and the grand total across calendar year and country.

5.1.2 CUBE Query 2: Sales by Customer Category and Calendar Year

Objective. Examine sales including tax and profit across customer categories and invoice years.

SQL Query.

```
SELECT CASE WHEN GROUPING(d.[Calendar Year]) = 1 THEN 'ALL YEARS'
           ELSE CAST(d.[Calendar Year] AS varchar(10)) END AS [Calendar Year],
       CASE WHEN GROUPING(cu.[Category]) = 1 THEN 'ALL CATEGORIES'
           ELSE cu.[Category] END AS [Customer Category],
       SUM(s.[Total Including Tax]) AS [Sales Including Tax],
       SUM(s.[Profit]) AS [Profit],
       GROUPING_ID(d.[Calendar Year], cu.[Category]) AS [Grouping ID]
FROM [Fact].[Sale] s
JOIN [Dimension].[Date] d ON s.[Invoice Date Key] = d.[Date]
JOIN [Dimension].[Customer] cu ON s.[Customer Key] = cu.[Customer Key]
GROUP BY CUBE(d.[Calendar Year], cu.[Category])
ORDER BY GROUPING_ID(d.[Calendar Year], cu.[Category]),
         d.[Calendar Year], cu.[Category];
```

Result. The query returned 15 rows. The following is the actual output, with monetary values shown to two decimal places.

Table 8: Actual output of CUBE Query 2 (15 rows)

Year	Category	Sales incl. tax	Profit	GID
2013	N/A	17,875,084.97	7,751,303.70	0
2013	Novelty Shop	34,688,187.67	15,017,048.55	0
2014	N/A	20,965,802.38	9,093,903.80	0
2014	Novelty Shop	36,453,114.51	15,734,558.65	0
2015	N/A	23,971,428.29	10,409,233.60	0
2015	Novelty Shop	38,118,792.52	16,548,367.05	0
2016	N/A	10,224,728.14	4,406,411.65	0
2016	Novelty Shop	15,746,300.97	6,768,353.90	0
2013	ALL CATEGORIES	52,563,272.64	22,768,352.25	1
2014	ALL CATEGORIES	57,418,916.89	24,828,462.45	1
2015	ALL CATEGORIES	62,090,220.81	26,957,600.65	1
2016	ALL CATEGORIES	25,971,029.11	11,174,765.55	1
ALL YEARS	N/A	73,037,043.78	31,660,852.75	2
ALL YEARS	Novelty Shop	125,006,395.67	54,068,328.15	2
ALL YEARS	ALL CATEGORIES	198,043,439.45	85,729,180.90	3

Interpretation. GID 0 gives the category-year detail, GID 1 gives each year's category subtotal, GID 2 gives each category across all years, and GID 3 gives the grand total. The two actual categories are N/A and Novelty Shop; Novelty Shop contributes 125,006,395.67 of the 198,043,439.45 total sales including tax.

5.1.3 CUBE Query 3: Orders by Stock Item Brand and Country

Objective. Compare ordered units and order value by the stock-item brand and delivery city country dimensions.

SQL Query.

```
SELECT CASE WHEN GROUPING(si.[Brand]) = 1 THEN 'ALL BRANDS'
           ELSE si.[Brand] END AS [Brand],
       CASE WHEN GROUPING(c.[Country]) = 1 THEN 'ALL COUNTRIES'
           ELSE c.[Country] END AS [Country],
       SUM(o.[Quantity]) AS [Units Ordered],
       SUM(o.[Total Including Tax]) AS [Order Value Including Tax],
       GROUPING_ID(si.[Brand], c.[Country]) AS [Grouping ID]
FROM [Fact].[Order] o
JOIN [Dimension].[Stock Item] si ON o.[Stock Item Key] = si.[Stock Item Key]
JOIN [Dimension].[City] c ON o.[City Key] = c.[City Key]
GROUP BY CUBE(si.[Brand], c.[Country])
ORDER BY GROUPING_ID(si.[Brand], c.[Country]), si.[Brand], c.[Country];
```

Result. The query returned 6 rows, shown completely below.

Table 9: Actual output of CUBE Query 3 (6 rows)

Brand	Country	Units	Order value	GID
N/A	United States	9,206,739	187,199,768.13	0
Northwind	United States	104,165	17,022,547.25	0
N/A	ALL COUNTRIES	9,206,739	187,199,768.13	1
Northwind	ALL COUNTRIES	104,165	17,022,547.25	1
ALL BRANDS	United States	9,310,904	204,222,315.38	2
ALL BRANDS	ALL COUNTRIES	9,310,904	204,222,315.38	3

Interpretation. The detailed brand-country rows show that all order rows are associated with the United States, while the brand subtotal rows aggregate that country away. The grand total is 9,310,904 units and 204,222,315.38 in order value including tax. This cube also exposes the limited but real brand variation in the inspected order data.

5.1.4 CUBE Query 4: Purchases by Supplier and Calendar Year

Objective. Summarize ordered and received quantities by purchase year and supplier to support supplier and time comparisons.

SQL Query.

```
SELECT CASE WHEN GROUPING(d.[Calendar Year]) = 1 THEN 'ALL YEARS'
      ELSE CAST(d.[Calendar Year] AS varchar(10)) END AS [Calendar Year],
CASE WHEN GROUPING(su.[Supplier]) = 1 THEN 'ALL SUPPLIERS'
      ELSE su.[Supplier] END AS [Supplier],
SUM(p.[Ordered Quantity]) AS [Ordered Quantity],
SUM(p.[Received Outers]) AS [Received Outers],
GROUPING_ID(d.[Calendar Year], su.[Supplier]) AS [Grouping ID]
FROM [Fact].[Purchase] p
JOIN [Dimension].[Date] d ON p.[Date Key] = d.[Date]
JOIN [Dimension].[Supplier] su ON p.[Supplier Key] = su.[Supplier Key]
GROUP BY CUBE(d.[Calendar Year], su.[Supplier])
ORDER BY GROUPING_ID(d.[Calendar Year], su.[Supplier]),
         d.[Calendar Year], su.[Supplier];
```

Result. The query returned 25 rows. The table presents the actual yearly supplier detail for the larger suppliers together with the all-supplier subtotals and grand total; the omitted detail rows are the actual small suppliers Contoso, Ltd., Graphic Design Institute, Northwind Electric Cars, The Phone Company, and A Datum Corporation.

Table 10: Representative actual output of CUBE Query 4 (25 rows returned)

Year	Supplier	Ordered	Received	GID
2013	Fabrikam, Inc.	7,118,315	593,797	0
2013	Litware, Inc.	1,400,592	82,494	0
2014	Fabrikam, Inc.	22,471,596	1,872,633	0
2014	Litware, Inc.	9,373,564	550,607	0
2015	Fabrikam, Inc.	39,609,324	3,300,777	0
2015	Litware, Inc.	23,741,191	1,309,350	0
2016	Fabrikam, Inc.	21,943,044	1,810,376	0
2016	Litware, Inc.	14,111,417	759,981	0
2013	ALL SUPPLIERS	8,523,392	680,641	1
2014	ALL SUPPLIERS	31,845,160	2,423,240	1
2015	ALL SUPPLIERS	63,350,515	4,610,127	1
2016	ALL SUPPLIERS	36,059,537	2,570,578	1
ALL YEARS	Fabrikam, Inc.	91,142,279	7,577,583	2
ALL YEARS	Litware, Inc.	48,626,764	2,702,432	2
ALL YEARS	ALL SUPPLIERS	139,778,604	10,284,586	3

Interpretation. GID 0 rows retain both supplier and year, GID 1 aggregates suppliers within each year, GID 2 aggregates years for each supplier, and GID 3 is the overall purchase total. Fabrikam, Inc. is the largest supplier by ordered quantity, while the cube makes the strong growth from 2013 to 2015 immediately visible in the yearly subtotals.

5.1.5 CUBE Query 5: Transactions by Type, Payment Method, and Calendar Year

Objective. Analyze transaction value across three dimensions: year, transaction type, and payment method.

SQL Query.

```
SELECT CASE WHEN GROUPING(d.[Calendar Year]) = 1 THEN 'ALL YEARS'
      ELSE CAST(d.[Calendar Year] AS varchar(10)) END AS [Calendar Year],
```

```

CASE WHEN GROUPING(tt.[Transaction Type]) = 1 THEN 'ALL TYPES'
      ELSE tt.[Transaction Type] END AS [Transaction Type],
CASE WHEN GROUPING(pm.[Payment Method]) = 1 THEN 'ALL METHODS'
      ELSE pm.[Payment Method] END AS [Payment Method],
SUM(t.[Total Including Tax]) AS [Transaction Value Including Tax],
GROUPING_ID(d.[Calendar Year], tt.[Transaction Type],
            pm.[Payment Method]) AS [Grouping ID]
FROM [Fact].[Transaction] t
JOIN [Dimension].[Date] d ON t.[Date Key] = d.[Date]
JOIN [Dimension].[Transaction Type] tt
      ON t.[Transaction Type Key] = tt.[Transaction Type Key]
LEFT JOIN [Dimension].[Payment Method] pm
      ON t.[Payment Method Key] = pm.[Payment Method Key]
GROUP BY CUBE(d.[Calendar Year], tt.[Transaction Type], pm.[Payment Method])
ORDER BY GROUPING_ID(d.[Calendar Year], tt.[Transaction Type],
                    pm.[Payment Method]), d.[Calendar Year],
                    tt.[Transaction Type], pm.[Payment Method];

```

Result. The query returned 55 rows. The following representative rows are copied from the actual output and include detail, intermediate subtotals, and the grand total.

Table 11: Representative actual output of CUBE Query 5 (55 rows returned)

Year	Type	Method	Value incl. tax	GID
2013	Customer Invoice	Unknown	52,563,272.64	0
2013	Customer Payment Received	EFT	-52,324,023.90	0
2013	Supplier Invoice	EFT	70,428,667.37	0
2013	Customer Invoice	ALL METHODS	52,563,272.64	1
2013	ALL TYPES	EFT	-51,719,300.08	2
2013	ALL TYPES	ALL METHODS	843,972.56	3
ALL YEARS	Customer Invoice	Unknown	198,043,439.45	4
ALL YEARS	ALL TYPES	EFT	-195,054,749.80	6
ALL YEARS	ALL TYPES	ALL METHODS	2,988,689.65	7

Interpretation. GID 0 represents year-type-method detail; GIDs 1, 2, and 4 aggregate one selected dimension at a time; GID 3 aggregates both type and method within a year; GID 6 aggregates year and type while retaining payment method; and GID 7 is the grand total. Negative values represent payment-outflow transaction rows in the fact data. The overall net transaction value is 2,988,689.65 after combining all types and methods.

5.1.6 CUBE Experiment Summary

Table 12 summarizes the five executed experiments and demonstrates that the CUBE operation was applied across several fact processes rather than repeated over one subject area.

Table 12: Summary of executed CUBE experiments

Query	Fact table	Dimensions	Measure	Purpose
1	Fact.Sale	Calendar Year, Country	Quantity, Profit	Geography and time sales comparison
2	Fact.Sale	Customer Category, Calendar Year	Total Including Tax, Profit	Customer-segment sales analysis
3	Fact.Order	Stock Item Brand, Country	Quantity, Total Including Tax	Product and geography order analysis
4	Fact.Purchase	Supplier, Calendar Year	Ordered Quantity, Received Outers	Supplier performance over time
5	Fact.Transaction	Calendar Year, Transaction Type, Payment Method	Total Including Tax	Financial flow analysis

5.2 ROLLUP Operation

ROLLUP generates hierarchical subtotals from the most detailed grouping level toward progressively higher aggregation levels and finally a grand total. The order of expressions inside ROLLUP matters: the first expression is retained longest, while later expressions are removed as the result rolls upward. The following five experiments use different fact processes and label generated subtotal rows with GROUPING() or GROUPING_ID().

5.2.1 ROLLUP Query 1: Geographic Sales Hierarchy

Objective. Roll sales units from city to state/province, country, and grand total for three verified United States city members.

```

SELECT CASE WHEN GROUPING(c.[Country])=1 THEN 'ALL COUNTRIES' ELSE c.[Country] END AS [Country],
       CASE WHEN GROUPING(c.[State Province])=1 THEN 'ALL STATES' ELSE c.[State Province] END AS [State
         Province],
       CASE WHEN GROUPING(c.[City])=1 THEN 'ALL CITIES' ELSE c.[City] END AS [City],
       SUM(s.[Quantity]) AS [Units Sold],
       GROUPING_ID(c.[Country],c.[State Province],c.[City]) AS [GID]
FROM [Fact].[Sale] s
JOIN [Dimension].[City] c ON s.[City Key]=c.[City Key]
WHERE c.[Country]='United States'
      AND ((c.[State Province]='Ohio' AND c.[City]='East Fultonham')
          OR (c.[State Province]='Wyoming' AND c.[City]='Sinclair')
          OR (c.[State Province]='Alaska' AND c.[City]='Akhiok'))
GROUP BY ROLLUP(c.[Country],c.[State Province],c.[City])
ORDER BY GROUPING_ID(c.[Country],c.[State Province],c.[City]),c.[State Province],c.[City];

```

Result. Eight rows were returned. The detailed city rows were Akhiok 30,999, East Fultonham 25,493, and Sinclair 30,114 units; state subtotals were 30,999, 25,493, and 30,114; the United States subtotal and grand total were both 86,606.

Table 13: ROLLUP Query 1 actual output

Country	State	City	Units	GID
United States	Alaska	Akhiok	30,999	0
United States	Ohio	East Fultonham	25,493	0
United States	Wyoming	Sinclair	30,114	0
United States	Alaska	ALL CITIES	30,999	1
United States	Ohio	ALL CITIES	25,493	1
United States	Wyoming	ALL CITIES	30,114	1
United States	ALL STATES	ALL CITIES	86,606	3
ALL COUNTRIES	ALL STATES	ALL CITIES	86,606	7

Interpretation. GID 0 is the city level, GID 1 is the state subtotal, GID 3 is the country subtotal, and GID 7 is the grand total. The hierarchy makes the geographic contribution of each selected city visible without mixing it with unrelated locations.

```

+ DW git:(main) x docker exec sqlserver /opt/mssql-tools18/bin/sqlcmd \
-S localhost \
-U sa \
-P 'StrongPass!123' \
-C \
--
END AS [Country],
CASE WHEN GROUPING(c.[State Province]) = 1
THEN 'ALL STATES'
ELSE c.[State Province]
END AS [State Province],
CASE WHEN GROUPING(c.[City]) = 1
THEN 'ALL CITIES'
ELSE c.[City]
END AS [City],
SUM(s.[Quantity]) AS [Units Sold],
GROUPING_ID(c.[Country], c.[State Province], c.[City]) AS [GID]
FROM [Fact].[Sale] s
JOIN [Dimension].[City] c
ON s.[City Key] = c.[City Key]
WHERE c.[Country] = 'United States'
      AND (
(c.[State Province] = 'Ohio' AND c.[City] = 'East Fultonham')
OR
(c.[State Province] = 'Wyoming' AND c.[City] = 'Sinclair')
OR
(c.[State Province] = 'Alaska' AND c.[City] = 'Akhiok')
)
GROUP BY ROLLUP(
c.[Country],
c.[State Province],
c.[City]
)
ORDER BY
GROUPING_ID(c.[Country], c.[State Province], c.[City]),
c.[State Province],
c.[City];
"
Country,State Province,City,Units Sold,GID
-----
United States,Alaska,Akhiok,30999,0
United States,Ohio,East Fultonham,25493,0
United States,Wyoming,Sinclair,30114,0
United States,Alaska,ALL CITIES,30999,1
United States,Ohio,ALL CITIES,25493,1
United States,Wyoming,ALL CITIES,30114,1
United States,ALL STATES,ALL CITIES,86606,3
ALL COUNTRIES,ALL STATES,ALL CITIES,86606,7
(8 rows affected)
+ DW git:(main) x

```

Figure 5: Execution of a ROLLUP query showing progressive aggregation from city to state, country, and grand total.

5.2.2 ROLLUP Query 2: Order Calendar Hierarchy

Objective. Roll order quantity from individual dates to the month, year, and grand total for the first three days of January 2015.

```
SELECT CASE WHEN GROUPING(d.[Calendar Year])=1 THEN 'ALL YEARS' ELSE CAST(d.[Calendar Year] AS varchar(10)
) END AS [Year],
       CASE WHEN GROUPING(d.[Calendar Month Label])=1 THEN 'ALL MONTHS' ELSE d.[Calendar Month Label] END
       AS [Month],
       CASE WHEN GROUPING(d.[Date])=1 THEN 'ALL DATES' ELSE CONVERT(varchar(10),d.[Date],23) END AS [Date],
       SUM(o.[Quantity]) AS [Units Ordered],
       GROUPING_ID(d.[Calendar Year],d.[Calendar Month Label],d.[Date]) AS [GID]
FROM [Fact].[Order] o
JOIN [Dimension].[Date] d ON o.[Order Date Key]=d.[Date]
WHERE d.[Date] BETWEEN '2015-01-01' AND '2015-01-03'
GROUP BY ROLLUP(d.[Calendar Year],d.[Calendar Month Label],d.[Date])
ORDER BY GROUPING_ID(d.[Calendar Year],d.[Calendar Month Label],d.[Date]),d.[Date];
```

Result. Six rows were returned: 12,764, 7,270, and 2,768 units on the three dates; 22,802 for the month, 22,802 for the year, and 22,802 for the grand total. The date rows are the detailed level and the final three rows demonstrate the progressive roll-up.

5.2.3 ROLLUP Query 3: Product Hierarchy in Orders

Objective. Roll ordered quantity from individual stock items to the verified Northwind brand. The query deliberately displays the ten largest detailed rows returned by its bounded sample query.

```
WITH r AS (
  SELECT CASE WHEN GROUPING(si.[Brand])=1 THEN 'ALL BRANDS' ELSE si.[Brand] END AS [Brand],
         CASE WHEN GROUPING(si.[Stock Item])=1 THEN 'ALL STOCK ITEMS' ELSE si.[Stock Item] END AS [Stock
         Item],
         SUM(o.[Quantity]) AS [Units Ordered],
         GROUPING_ID(si.[Brand],si.[Stock Item]) AS [GID]
  FROM [Fact].[Order] o
  JOIN [Dimension].[Stock Item] si ON o.[Stock Item Key]=si.[Stock Item Key]
  WHERE si.[Brand]='Northwind'
  GROUP BY ROLLUP(si.[Brand],si.[Stock Item])
)
SELECT TOP 10 * FROM r ORDER BY [GID],[Units Ordered] DESC;
```

Result. The query returned ten displayed rows. The largest detailed values were 6,110, 5,960, 5,919, 5,888, and 5,881 units for the first five readable Northwind stock items; the query is bounded to keep the report readable.

Interpretation. The stock-item rows are the detailed product level, while the omitted higher-level rows in the bounded display are generated by the same ROLLUP result. This is a product hierarchy rather than a repeated time or geography analysis.

5.2.4 ROLLUP Query 4: Customer Buying Group Hierarchy

Objective. Roll sales including tax from customer to category and buying group for two verified buying groups.

```
WITH r AS (
  SELECT CASE WHEN GROUPING(cu.[Buying Group])=1 THEN 'ALL BUYING GROUPS' ELSE cu.[Buying Group] END AS [
  Buying Group],
         CASE WHEN GROUPING(cu.[Category])=1 THEN 'ALL CATEGORIES' ELSE cu.[Category] END AS [Category],
         CASE WHEN GROUPING(cu.[Customer])=1 THEN 'ALL CUSTOMERS' ELSE cu.[Customer] END AS [Customer],
         SUM(s.[Total Including Tax]) AS [Sales Including Tax],
         GROUPING_ID(cu.[Buying Group],cu.[Category],cu.[Customer]) AS [GID]
  FROM [Fact].[Sale] s
  JOIN [Dimension].[Customer] cu ON s.[Customer Key]=cu.[Customer Key]
  WHERE cu.[Buying Group] IN ('Tailspin Toys','Wingtip Toys')
  GROUP BY ROLLUP(cu.[Buying Group],cu.[Category],cu.[Customer])
)
SELECT TOP 12 * FROM r ORDER BY [GID],[Sales Including Tax] DESC;
```

Result. The bounded query returned twelve detailed rows. The largest displayed customer totals were 438,689.81, 427,445.57, 420,001.08, 414,934.18, and 408,524.95. All displayed detailed rows belong to the verified Novelty Shop category.

Interpretation. The detailed customer level can be rolled to category and then buying group. GID 0 identifies customers; higher GIDs identify the category and buying-group subtotal levels. The TOP clause limits the displayed portion, not the validity of the hierarchy.

5.2.5 ROLLUP Query 5: Movement Transaction Type to Stock Item

Objective. Roll movement quantity from stock item to transaction type for the verified Stock Issue movement process.

```
WITH r AS (
  SELECT CASE WHEN GROUPING(tt.[Transaction Type])=1 THEN 'ALL TYPES' ELSE tt.[Transaction Type] END AS [
  Transaction Type],
         CASE WHEN GROUPING(si.[Stock Item])=1 THEN 'ALL STOCK ITEMS' ELSE si.[Stock Item] END AS [Stock
         Item],
         SUM(m.[Quantity]) AS [Moved Quantity],
         GROUPING_ID(tt.[Transaction Type],si.[Stock Item]) AS [GID]
```

```

FROM [Fact].[Movement] m
JOIN [Dimension].[Transaction Type] tt ON m.[Transaction Type Key]=tt.[Transaction Type Key]
JOIN [Dimension].[Stock Item] si ON m.[Stock Item Key]=si.[Stock Item Key]
WHERE tt.[Transaction Type]='Stock Issue'
GROUP BY ROLLUP(tt.[Transaction Type],si.[Stock Item])
)
SELECT TOP 10 * FROM r ORDER BY [GID],[Moved Quantity] DESC;

```

Result. Ten detail rows were returned by the bounded query. The displayed quantities were negative, including -5,373, -5,402, -5,404, -5,426, and -5,464 for the largest displayed stock-issue rows.

Interpretation. Negative quantities are consistent with issue movements in this fact table. The operation supports stock-item detail followed by transaction-type aggregation, with the bounded result preserving readable business labels.

Table 14: ROLLUP experiment summary

Query	Fact	Hierarchy	Measure	Purpose
1	Fact.Sale	Country → State → City	Quantity	Geographic subtotals
2	Fact.Order	Year → Month → Date	Quantity	Calendar aggregation
3	Fact.Order	Brand → Stock Item	Quantity	Product aggregation
4	Fact.Sale	Buying Group → Category → Customer	Total Including Tax	Customer hierarchy
5	Fact.Movement	Transaction Type → Stock Item	Quantity	Inventory issue hierarchy

5.3 Drill-Down Operation

Drill-down moves from summarized data to progressively finer levels of detail. Unlike a slice, it changes the grouping granularity: the same measure is first viewed at a high level, then at a lower level, and finally at an even more detailed level where appropriate.

5.3.1 Drill-Down Experiment 1: Sales Year to Month to Date

Objective. Start with annual sales, refine to the twelve calendar months of 2015, and finally inspect the first three January dates. The three executed queries were the following.

```

-- Level 1: year
SELECT d.[Calendar Year] AS [Year], SUM(s.[Total Including Tax]) AS [Sales Including Tax]
FROM [Fact].[Sale] s JOIN [Dimension].[Date] d ON s.[Invoice Date Key]=d.[Date]
GROUP BY d.[Calendar Year] ORDER BY d.[Calendar Year];

-- Level 2: year and month
SELECT d.[Calendar Year] AS [Year], d.[Calendar Month Label] AS [Month],
       SUM(s.[Total Including Tax]) AS [Sales Including Tax]
FROM [Fact].[Sale] s JOIN [Dimension].[Date] d ON s.[Invoice Date Key]=d.[Date]
WHERE d.[Calendar Year]=2015
GROUP BY d.[Calendar Year],d.[Calendar Month Label],d.[Calendar Month Number]
ORDER BY d.[Calendar Month Number];

-- Level 3: year, month, and date
SELECT d.[Calendar Year] AS [Year], d.[Calendar Month Label] AS [Month], d.[Date],
       SUM(s.[Total Including Tax]) AS [Sales Including Tax]
FROM [Fact].[Sale] s JOIN [Dimension].[Date] d ON s.[Invoice Date Key]=d.[Date]
WHERE d.[Date] BETWEEN '2015-01-01' AND '2015-01-03'
GROUP BY d.[Calendar Year],d.[Calendar Month Label],d.[Date]
ORDER BY d.[Date];

```

Actual results. Level 1 returned 2013: 52,563,272.64, 2014: 57,418,916.89, 2015: 62,090,220.81, and 2016: 25,971,029.11. Level 2 returned 12 monthly rows; January, February, and March 2015 were 5,061,954.63, 4,824,617.67, and 5,207,351.93. Level 3 returned January 1–3 values of 260,229.32, 150,455.56, and 51,394.37.

Interpretation. The analysis moves from four annual rows to twelve monthly rows for one year and then three daily rows. This is a change in granularity, not merely a WHERE restriction.

5.3.2 Drill-Down Experiment 2: Country to State to City

Objective. Refine sales quantity from country to state and then to the largest cities in Ohio.

```

-- Level 1: country
SELECT c.[Country],SUM(s.[Quantity]) AS [Units Sold]
FROM [Fact].[Sale] s JOIN [Dimension].[City] c ON s.[City Key]=c.[City Key]
GROUP BY c.[Country] ORDER BY [Units Sold] DESC;

-- Level 2: country and state (top five displayed)
WITH r AS (SELECT c.[Country],c.[State Province],SUM(s.[Quantity]) AS [Units Sold]

```

```

DM git:(main) x docker exec sqlserver /opt/mssql-tools18/bin/sqlcmd \
-S localhost \
-U sa \
-P 'StrongPass!123' \
-C \
-d WideWorldImportersDW \
-s "," \
-W \
-Q "
SELECT
    d.[Calendar Year] AS [Year],
    CAST(SUM(s.[Total Including Tax]) AS decimal(18,2)) AS [Sales Including Tax]
FROM [Fact].[Sale] s
JOIN [Dimension].[Date] d
    ON s.[Invoice Date Key] = d.[Date]
WHERE d.[Calendar Year] = 2015
GROUP BY
    d.[Calendar Year],
    d.[Calendar Month Label],
    d.[Calendar Month Number]
ORDER BY d.[Calendar Month Number];
"
Year,Sales Including Tax
-----
2013,52563272.64
2014,57418916.89
2015,62090220.81
2016,25971029.11
(4 rows affected)
DM git:(main) x

```

Figure 6: Drill-down execution at the annual level for the Year → Month → Date sales analysis.

```

DM git:(main) x docker exec sqlserver /opt/mssql-tools18/bin/sqlcmd \
-S localhost \
-U sa \
-P 'StrongPass!123' \
-C \
-d WideWorldImportersDW \
-s "," \
-W \
-Q "
SELECT
    d.[Calendar Year] AS [Year],
    d.[Calendar Month Label] AS [Month],
    CAST(SUM(s.[Total Including Tax]) AS decimal(18,2)) AS [Sales Including Tax]
FROM [Fact].[Sale] s
JOIN [Dimension].[Date] d
    ON s.[Invoice Date Key] = d.[Date]
WHERE d.[Calendar Year] = 2015
GROUP BY
    d.[Calendar Year],
    d.[Calendar Month Label],
    d.[Calendar Month Number]
ORDER BY d.[Calendar Month Number];
"
Year,Month,Sales Including Tax
-----
2015,CY2015-Jan,5061954.63
2015,CY2015-Feb,4824617.67
2015,CY2015-Mar,5207351.93
2015,CY2015-Apr,5834255.09
2015,CY2015-May,5152840.72
2015,CY2015-Jun,5193217.15
2015,CY2015-Jul,5929023.38
2015,CY2015-Aug,4528888.44
2015,CY2015-Sep,5361990.68
2015,CY2015-Oct,5165857.34
2015,CY2015-Nov,4702590.28
2015,CY2015-Dec,5127633.50
(12 rows affected)
DM git:(main) x

```

Figure 7: Drill-down execution at the monthly level for the Year → Month → Date sales analysis.

```

DM git:(main) x docker exec sqlserver /opt/mssql-tools18/bin/sqlcmd \
-S localhost \
-U sa \
-P 'StrongPass!123' \
-C \
-d WideWorldImportersDW \
-s "," \
-W \
-Q "
SELECT
    d.[Calendar Year] AS [Year],
    d.[Calendar Month Label] AS [Month],
    CONVERT(varchar(10), d.[Date], 23) AS [Date],
    CAST(SUM(s.[Total Including Tax]) AS decimal(18,2)) AS [Sales Including Tax]
FROM [Fact].[Sale] s
JOIN [Dimension].[Date] d
    ON s.[Invoice Date Key] = d.[Date]
WHERE d.[Date] BETWEEN '2015-01-01' AND '2015-01-03'
GROUP BY
    d.[Calendar Year],
    d.[Calendar Month Label],
    d.[Date]
ORDER BY d.[Date];
"
Year,Month,Date,Sales Including Tax
-----
2015,CY2015-Jan,2015-01-01,260229.32
2015,CY2015-Jan,2015-01-02,150455.56
2015,CY2015-Jan,2015-01-03,51394.37
(3 rows affected)
DM git:(main) x

```

Figure 8: Drill-down execution at the daily level for the Year → Month → Date sales analysis.

```

FROM [Fact].[Sale] s JOIN [Dimension].[City] c ON s.[City Key]=c.[City Key]
WHERE c.[Country]='United States' GROUP BY c.[Country],c.[State Province])
SELECT TOP 5 * FROM r ORDER BY [Units Sold] DESC;

-- Level 3: country, state, and city (top five Ohio cities displayed)
WITH r AS (SELECT c.[Country],c.[State Province],c.[City],SUM(s.[Quantity]) AS [Units Sold]
FROM [Fact].[Sale] s JOIN [Dimension].[City] c ON s.[City Key]=c.[City Key]
WHERE c.[Country]='United States' AND c.[State Province]='Ohio'
GROUP BY c.[Country],c.[State Province],c.[City])
SELECT TOP 5 * FROM r ORDER BY [Units Sold] DESC;

```

Actual results. The country level returned United States with 8,950,628 units. The top state values were Texas 627,272, Pennsylvania 510,038, California 461,064, New York 452,684, and Ohio 289,975. The top Ohio cities were East Fultonham 25,493, Bourneville 18,292, Lucasville 16,526, Obetz 15,413, and North Eaton 15,296.

5.3.3 Drill-Down Experiment 3: Customer Category to Customer

Objective. Move from category-level sales to customer-level sales.

```

-- Level 1: category
SELECT cu.[Category],SUM(s.[Total Including Tax]) AS [Sales Including Tax]
FROM [Fact].[Sale] s JOIN [Dimension].[Customer] cu ON s.[Customer Key]=cu.[Customer Key]
GROUP BY cu.[Category] ORDER BY [Sales Including Tax] DESC;

-- Level 2: category and customer (top five displayed)
WITH r AS (SELECT cu.[Category],cu.[Customer],SUM(s.[Total Including Tax]) AS [Sales Including Tax]
FROM [Fact].[Sale] s JOIN [Dimension].[Customer] cu ON s.[Customer Key]=cu.[Customer Key]
GROUP BY cu.[Category],cu.[Customer])
SELECT TOP 5 * FROM r ORDER BY [Sales Including Tax] DESC;

```

Actual results. Category totals were Novelty Shop 125,006,395.67 and N/A 73,037,043.78. The detailed sample began with Unknown 73,037,043.78, followed by Tailspin Toys (Inguadona, MN) 438,689.81, Tailspin Toys (Minidoka, ID) 427,445.57, Wingtip Toys (Sarversville, PA) 420,001.08, and Tailspin Toys (Long Meadow, MD) 414,934.18.

5.3.4 Drill-Down Experiment 4: Product Brand to Stock Item

Objective. Refine order quantity from brand to individual stock item for the Northwind brand.

```

-- Level 1: brand
SELECT si.[Brand],SUM(o.[Quantity]) AS [Units Ordered]
FROM [Fact].[Order] o JOIN [Dimension].[Stock Item] si ON o.[Stock Item Key]=si.[Stock Item Key]
GROUP BY si.[Brand] ORDER BY [Units Ordered] DESC;

-- Level 2: brand and stock item
WITH r AS (SELECT si.[Brand],si.[Stock Item],SUM(o.[Quantity]) AS [Units Ordered]
FROM [Fact].[Order] o JOIN [Dimension].[Stock Item] si ON o.[Stock Item Key]=si.[Stock Item Key]
WHERE si.[Brand]='Northwind' GROUP BY si.[Brand],si.[Stock Item])
SELECT TOP 5 * FROM r ORDER BY [Units Ordered] DESC;

```

Actual results. Brand totals were N/A 9,206,739 and Northwind 104,165 units. The top Northwind stock items were the black ride-on vintage coupe 6,110, red remote-control vintage coupe 5,960, black remote-control vintage coupe 5,919, black remote-control sedan 5,888, and blue ride-on sedan 5,881.

Table 15: Drill-down experiment summary

Experiment	Fact	Hierarchy	Starting level	Final level	Measure
1	Fact.Sale	Year → Month → Date	Year	Date	Total Including Tax
2	Fact.Sale	Country → State → City	Country	City	Quantity
3	Fact.Sale	Category → Customer	Category	Customer	Total Including Tax
4	Fact.Order	Brand → Stock Item	Brand	Stock Item	Quantity

5.4 Slice Operation

A slice fixes one dimension to a single verified value, reducing a multidimensional cube to a smaller analytical view. Each query below explicitly identifies its sliced dimension and uses a different fact or analytical subject where practical.

5.4.1 Slice Query 1: Calendar Year 2015

Objective and slice condition. Fix Dimension.Date.Calendar Year = 2015 and measure sales units and profit.

```
SELECT d.[Calendar Year] AS [Year],SUM(s.[Quantity]) AS [Units Sold],SUM(s.[Profit]) AS [Profit]
FROM [Fact].[Sale] s JOIN [Dimension].[Date] d ON s.[Invoice Date Key]=d.[Date]
WHERE d.[Calendar Year]=2015 GROUP BY d.[Calendar Year];
```

Actual output. 2015, 2,740,266 units, profit 26,957,600.65. **Interpretation.** All other date members are removed from the view.

5.4.2 Slice Query 2: Country United States

Objective and slice condition. Fix Dimension.City.Country = United States and compare order quantity by brand.

```
SELECT si.[Brand],SUM(o.[Quantity]) AS [Units Ordered]
FROM [Fact].[Order] o JOIN [Dimension].[City] c ON o.[City Key]=c.[City Key]
JOIN [Dimension].[Stock Item] si ON o.[Stock Item Key]=si.[Stock Item Key]
WHERE c.[Country]='United States' GROUP BY si.[Brand] ORDER BY si.[Brand];
```

Actual output. N/A: 9,206,739; Northwind: 104,165 units. **Interpretation.** The slice restricts geography while retaining product-brand variation.

5.4.3 Slice Query 3: Supplier Fabrikam, Inc.

Objective and slice condition. Fix Dimension.Supplier.Supplier = Fabrikam, Inc. and show ordered quantity by year.

```
SELECT d.[Calendar Year] AS [Year],SUM(p.[Ordered Quantity]) AS [Ordered Quantity]
FROM [Fact].[Purchase] p JOIN [Dimension].[Date] d ON p.[Date Key]=d.[Date]
JOIN [Dimension].[Supplier] su ON p.[Supplier Key]=su.[Supplier Key]
WHERE su.[Supplier]='Fabrikam, Inc.' GROUP BY d.[Calendar Year] ORDER BY d.[Calendar Year];
```

Actual output. 2013: 7,118,315; 2014: 22,471,596; 2015: 39,609,324; 2016: 21,943,044. **Interpretation.** The supplier slice exposes its time profile without other suppliers.

5.4.4 Slice Query 4: Customer Invoice Transactions

Objective and slice condition. Fix Dimension.Transaction Type.Transaction Type = Customer Invoice and aggregate transaction value by year.

```
SELECT d.[Calendar Year] AS [Year],SUM(t.[Total Including Tax]) AS [Transaction Value Including Tax]
FROM [Fact].[Transaction] t JOIN [Dimension].[Date] d ON t.[Date Key]=d.[Date]
JOIN [Dimension].[Transaction Type] tt ON t.[Transaction Type Key]=tt.[Transaction Type Key]
WHERE tt.[Transaction Type]='Customer Invoice' GROUP BY d.[Calendar Year] ORDER BY d.[Calendar Year];
```

Actual output. 2013: 52,563,272.64; 2014: 57,418,916.89; 2015: 62,090,220.81; 2016: 25,971,029.11. **Interpretation.** The slice isolates invoice inflows from payments and supplier transactions.

5.4.5 Slice Query 5: Stock Issue Movements

Objective and slice condition. Fix Dimension.Transaction Type.Transaction Type = Stock Issue and aggregate moved quantity by year.

```
SELECT d.[Calendar Year] AS [Year],SUM(m.[Quantity]) AS [Moved Quantity]
FROM [Fact].[Movement] m JOIN [Dimension].[Date] d ON m.[Date Key]=d.[Date]
JOIN [Dimension].[Transaction Type] tt ON m.[Transaction Type Key]=tt.[Transaction Type Key]
WHERE tt.[Transaction Type]='Stock Issue' GROUP BY d.[Calendar Year] ORDER BY d.[Calendar Year];
```

Actual output. 2013: -2,401,657; 2014: -2,567,401; 2015: -2,740,266; 2016: -1,241,304. **Interpretation.** The negative yearly totals are the actual sign convention for stock-issue movements in this fact.

Table 16: Slice experiment summary

Query	Fact	Sliced dimension	Selected value	Measure
1	Fact.Sale	Calendar Year	2015	Quantity, Profit
2	Fact.Order	Country	United States	Quantity
3	Fact.Purchase	Supplier	Fabrikam, Inc.	Ordered Quantity
4	Fact.Transaction	Transaction Type	Customer Invoice	Total Including Tax
5	Fact.Movement	Transaction Type	Stock Issue	Quantity

5.5 Dice Operation

Dice selects a subcube by applying conditions to multiple dimensions or ranges simultaneously. Unlike a slice, each query below constrains at least two dimensions, and the final query constrains three.

5.5.1 Dice Query 1: 2015 United States Sales

Dice dimensions and conditions. Calendar Year = 2015; Country = United States.

```
SELECT d.[Calendar Year] AS [Year],c.[Country],SUM(s.[Total Including Tax]) AS [Sales Including Tax]
FROM [Fact].[Sale] s JOIN [Dimension].[Date] d ON s.[Invoice Date Key]=d.[Date]
JOIN [Dimension].[City] c ON s.[City Key]=c.[City Key]
WHERE d.[Calendar Year]=2015 AND c.[Country]='United States'
GROUP BY d.[Calendar Year],c.[Country];
```

Actual output. 2015, United States, 62,090,220.81. This is a two-dimensional time-geography subcube.

5.5.2 Dice Query 2: Novelty Shop Profit in 2014–2015

Dice dimensions and conditions. Calendar Year BETWEEN 2014 AND 2015; Customer Category = Novelty Shop.

```
SELECT d.[Calendar Year] AS [Year],cu.[Category],SUM(s.[Profit]) AS [Profit]
FROM [Fact].[Sale] s JOIN [Dimension].[Date] d ON s.[Invoice Date Key]=d.[Date]
JOIN [Dimension].[Customer] cu ON s.[Customer Key]=cu.[Customer Key]
WHERE d.[Calendar Year] BETWEEN 2014 AND 2015 AND cu.[Category]='Novelty Shop'
GROUP BY d.[Calendar Year],cu.[Category] ORDER BY d.[Calendar Year];
```

Actual output. 2014: 15,734,558.65; 2015: 16,548,367.05. The dice retains only the selected time range and category.

5.5.3 Dice Query 3: 2015 Fabrikam and Litware Purchases

Dice dimensions and conditions. Calendar Year = 2015; Supplier IN (Fabrikam, Inc., Litware, Inc.).

```
SELECT d.[Calendar Year] AS [Year],su.[Supplier],SUM(p.[Ordered Quantity]) AS [Ordered Quantity]
FROM [Fact].[Purchase] p JOIN [Dimension].[Date] d ON p.[Date Key]=d.[Date]
JOIN [Dimension].[Supplier] su ON p.[Supplier Key]=su.[Supplier Key]
WHERE d.[Calendar Year]=2015 AND su.[Supplier] IN ('Fabrikam, Inc.','Litware, Inc.')
GROUP BY d.[Calendar Year],su.[Supplier] ORDER BY [Ordered Quantity] DESC;
```

Actual output. Fabrikam, Inc.: 39,609,324; Litware, Inc.: 23,741,191 ordered units.

5.5.4 Dice Query 4: United States Northwind Orders

Dice dimensions and conditions. Country = United States; Stock Item Brand = Northwind.

```
SELECT c.[Country],si.[Brand],SUM(o.[Total Including Tax]) AS [Order Value Including Tax]
FROM [Fact].[Order] o JOIN [Dimension].[City] c ON o.[City Key]=c.[City Key]
JOIN [Dimension].[Stock Item] si ON o.[Stock Item Key]=si.[Stock Item Key]
WHERE c.[Country]='United States' AND si.[Brand]='Northwind'
GROUP BY c.[Country],si.[Brand];
```

Actual output. United States, Northwind, 17,022,547.25. The result is a product-geography subcube.

5.5.5 Dice Query 5: 2015 EFT Supplier Invoices

Dice dimensions and conditions. Calendar Year = 2015; Transaction Type = Supplier Invoice; Payment Method = EFT.

```
SELECT d.[Calendar Year] AS [Year],tt.[Transaction Type],pm.[Payment Method],
SUM(t.[Total Including Tax]) AS [Transaction Value Including Tax]
FROM [Fact].[Transaction] t JOIN [Dimension].[Date] d ON t.[Date Key]=d.[Date]
JOIN [Dimension].[Transaction Type] tt ON t.[Transaction Type Key]=tt.[Transaction Type Key]
LEFT JOIN [Dimension].[Payment Method] pm ON t.[Payment Method Key]=pm.[Payment Method Key]
WHERE d.[Calendar Year]=2015 AND tt.[Transaction Type]='Supplier Invoice' AND pm.[Payment Method]='EFT'
GROUP BY d.[Calendar Year],tt.[Transaction Type],pm.[Payment Method];
```

Actual output. 2015, Supplier Invoice, EFT, 484,868,081.11. The three restrictions select a small financial subcube.

Table 17: Dice experiment summary

Query	Fact	Dice dimensions	Conditions	Measure	
1	Fact.Sale	Year, Country	2015; United States	Total Tax	Including
2	Fact.Sale	Year, Category	2014–2015; Novelty Shop	Profit	
3	Fact.Purchase	Year, Supplier	2015; Fabrikam or Litware	Ordered Quantity	
4	Fact.Order	Country, Brand	United States; Northwind	Total Tax	Including
5	Fact.Transaction	Year, Type, Method	2015; Supplier Invoice; EFT	Total Tax	Including

```

DM git:(main) x docker exec sqlserver /opt/mssql-tools18/bin/sqlcmd \
-S localhost \
-U sa \
-P 'StrongPass!123' \
-C \
-d WideWorldImportersDW \
-s "," \
-W \
-Q "
SELECT
    d.[Calendar Year] AS [Year],
    tt.[Transaction Type],
    pm.[Payment Method],
    CAST(SUM(t.[Total Including Tax]) AS decimal(18,2)) AS [Transaction Value]
FROM [Fact].[Transaction] t
JOIN [Dimension].[Date] d
    ON t.[Date Key] = d.[Date]
JOIN [Dimension].[Transaction Type] tt
    ON t.[Transaction Type Key] = tt.[Transaction Type Key]
LEFT JOIN [Dimension].[Payment Method] pm
    ON t.[Payment Method Key] = pm.[Payment Method Key]
WHERE d.[Calendar Year] = 2015
    AND tt.[Transaction Type] = 'Supplier Invoice'
    AND pm.[Payment Method] = 'EFT'
GROUP BY
    d.[Calendar Year],
    tt.[Transaction Type],
    pm.[Payment Method];
"
Year,Transaction Type,Payment Method,Transaction Value
-----,-----,-----,-----
2015,Supplier Invoice,EFT,484868081.11

(1 rows affected)
DM git:(main) x

```

Figure 9: Execution of a DICE query selecting a financial subcube using year, transaction type, and payment method.

5.6 Pivot Operation

The SQL Server PIVOT operator rotates an aggregated dimension into columns. The values placed in each IN list were verified against the live dimension/date data before execution. Each result below keeps the row dimension readable and the pivoted columns small.

5.6.1 Pivot Query 1: Sales by Country Across Years

Objective. Display annual sales including tax as year columns for each country.

```
SELECT [Country],[2013],[2014],[2015],[2016]
FROM (SELECT c.[Country],d.[Calendar Year],s.[Total Including Tax] AS [Value]
      FROM [Fact].[Sale] s JOIN [Dimension].[Date] d ON s.[Invoice Date Key]=d.[Date]
      JOIN [Dimension].[City] c ON s.[City Key]=c.[City Key]) src
PIVOT (SUM([Value]) FOR [Calendar Year] IN ([2013],[2014],[2015],[2016])) p
ORDER BY [Country];
```

Actual output. United States: 52,563,272.64; 57,418,916.89; 62,090,220.81; 25,971,029.11 for 2013–2016. The rows are countries and the columns are years.

```
DM git:(main) x docker exec sqlserver /opt/mssql-tools18/bin/sqlcmd \
-S localhost \
-U sa \
-P 'StrongPass!123' \
-C \
-d WideWorldImportersDW \
-s "," \
-W \
-Q "
SELECT
[Country],
[2013],
[2014],
[2015],
[2016]
FROM (
SELECT
c.[Country],
d.[Calendar Year],
s.[Total Including Tax] AS [Value]
FROM [Fact].[Sale] s
JOIN [Dimension].[Date] d
ON s.[Invoice Date Key] = d.[Date]
JOIN [Dimension].[City] c
ON s.[City Key] = c.[City Key]
) src
PIVOT (
SUM([Value])
FOR [Calendar Year] IN ([2013],[2014],[2015],[2016])
) p
ORDER BY [Country];
"
Country,2013,2014,2015,2016
-----,-----,-----,-----
United States,52563272.64,57418916.89,62090220.81,25971029.11

(1 rows affected)
DM git:(main) x
```

Figure 10: Execution of a PIVOT query showing annual sales values as separate year columns.

5.6.2 Pivot Query 2: Profit by Customer Category Across Years

Objective. Compare category profit year by year.

```
SELECT [Category],[2013],[2014],[2015],[2016]
FROM (SELECT cu.[Category],d.[Calendar Year],s.[Profit] AS [Value]
      FROM [Fact].[Sale] s JOIN [Dimension].[Date] d ON s.[Invoice Date Key]=d.[Date]
      JOIN [Dimension].[Customer] cu ON s.[Customer Key]=cu.[Customer Key]) src
PIVOT (SUM([Value]) FOR [Calendar Year] IN ([2013],[2014],[2015],[2016])) p
ORDER BY [Category];
```

Actual output. N/A: 7,751,303.70; 9,093,903.80; 10,409,233.60; 4,406,411.65; Novelty Shop: 15,017,048.55; 15,734,558.65; 16,548,367.05; 6,768,353.90. The columns make year-over-year category comparison direct.

5.6.3 Pivot Query 3: Ordered Quantity by Brand Across Years

Objective. Pivot order quantity by the two verified stock-item brands.

```
SELECT [Brand], [2013], [2014], [2015], [2016]
FROM (SELECT si.[Brand], d.[Calendar Year], o.[Quantity] AS [Value]
      FROM [Fact].[Order] o JOIN [Dimension].[Date] d ON o.[Order Date Key]=d.[Date]
      JOIN [Dimension].[Stock Item] si ON o.[Stock Item Key]=si.[Stock Item Key]) src
PIVOT (SUM([Value]) FOR [Calendar Year] IN ([2013],[2014],[2015],[2016])) p
ORDER BY [Brand];
```

Actual output. N/A: 2,445,963; 2,642,839; 2,840,165; 1,277,772; Northwind: 27,690; 30,134; 33,085; 13,256. The row dimension is Brand and the pivoted dimension is Calendar Year.

5.6.4 Pivot Query 4: Purchases by Supplier Across Years

Objective. Compare ordered quantity for Fabrikam, Inc. and Litware, Inc. across years.

```
SELECT [Supplier], [2013], [2014], [2015], [2016]
FROM (SELECT su.[Supplier], d.[Calendar Year], p.[Ordered Quantity] AS [Value]
      FROM [Fact].[Purchase] p JOIN [Dimension].[Date] d ON p.[Date Key]=d.[Date]
      JOIN [Dimension].[Supplier] su ON p.[Supplier Key]=su.[Supplier Key]
      WHERE su.[Supplier] IN ('Fabrikam, Inc.', 'Litware, Inc.')) src
PIVOT (SUM([Value]) FOR [Calendar Year] IN ([2013],[2014],[2015],[2016])) p
ORDER BY [Supplier];
```

Actual output. Fabrikam, Inc.: 7,118,315; 22,471,596; 39,609,324; 21,943,044. Litware, Inc.: 1,400,592; 9,373,564; 23,741,191; 14,111,417.

5.6.5 Pivot Query 5: Transactions by Type Across Payment Methods

Objective. Rotate payment methods into columns for transaction-type totals.

```
SELECT [Transaction Type], [EFT], [Unknown]
FROM (SELECT tt.[Transaction Type], pm.[Payment Method], t.[Total Including Tax] AS [Value]
      FROM [Fact].[Transaction] t
      JOIN [Dimension].[Transaction Type] tt ON t.[Transaction Type Key]=tt.[Transaction Type Key]
      LEFT JOIN [Dimension].[Payment Method] pm ON t.[Payment Method Key]=pm.[Payment Method Key]
      WHERE pm.[Payment Method] IN ('EFT', 'Unknown')) src
PIVOT (SUM([Value]) FOR [Payment Method] IN ([EFT],[Unknown])) p
ORDER BY [Transaction Type];
```

Actual output. Customer Invoice had Unknown 198,043,439.45; Customer Payment Received had EFT -197,776,428.01; Supplier Invoice had EFT 1,085,993,513.06; Supplier Payment Issued had EFT -1,083,271,834.85. The NULL cells in the pivot are genuine absence of that payment-method combination.

Table 18: Pivot experiment summary

Query	Fact	Row dimension	Pivoted dimension	Measure
1	Fact.Sale	Country	Calendar Year	Total Including Tax
2	Fact.Sale	Customer Category	Calendar Year	Profit
3	Fact.Order	Brand	Calendar Year	Quantity
4	Fact.Purchase	Supplier	Calendar Year	Ordered Quantity
5	Fact.Transaction	Transaction Type	Payment Method	Total Including Tax

5.7 Additional OLAP Operations

5.7.1 GROUPING SETS

GROUPING SETS explicitly names the aggregation combinations to calculate, rather than requesting every combination generated by CUBE or only the hierarchy implied by ROLLUP. SQL Server returns the union of the specified grouping sets. The following three verified queries select complementary combinations.

Grouping Sets Query 1. Sales by country and year, country only, year only, and the grand total.

```
SELECT CASE WHEN GROUPING(c.[Country])=1 THEN 'ALL COUNTRIES' ELSE c.[Country] END AS [Country],
       CASE WHEN GROUPING(d.[Calendar Year])=1 THEN 'ALL YEARS' ELSE CAST(d.[Calendar Year] AS varchar(10))
       END AS [Year],
       SUM(s.[Total Including Tax]) AS [Sales Including Tax],
       GROUPING_ID(c.[Country], d.[Calendar Year]) AS [GID]
FROM [Fact].[Sale] s JOIN [Dimension].[City] c ON s.[City Key]=c.[City Key]
JOIN [Dimension].[Date] d ON s.[Invoice Date Key]=d.[Date]
GROUP BY GROUPING SETS ((c.[Country], d.[Calendar Year]), (c.[Country]), (d.[Calendar Year]), ());
```

Actual output. The query returned 10 rows. Year detail values were 52,563,272.64, 57,418,916.89, 62,090,220.81, and 25,971,029.11; the grand total was 198,043,439.45.

Grouping Sets Query 2. Supplier totals and the overall purchase total.

```
SELECT su.[Supplier],SUM(p.[Ordered Quantity]) AS [Ordered Quantity]
FROM [Fact].[Purchase] p JOIN [Dimension].[Supplier] su ON p.[Supplier Key]=su.[Supplier Key]
GROUP BY GROUPING SETS ((su.[Supplier]),());
```

Actual output. Eight rows were returned. Fabrikam, Inc. had 91,142,279 and Litware, Inc. had 48,626,764; the grand total row was 139,778,604.

Grouping Sets Query 3. Transaction type/payment-method detail, transaction-type subtotals, and the grand total.

```
SELECT tt.[Transaction Type],pm.[Payment Method],SUM(t.[Total Including Tax]) AS [Transaction Value Including Tax]
FROM [Fact].[Transaction] t JOIN [Dimension].[Transaction Type] tt ON t.[Transaction Type Key]=tt.[Transaction Type Key]
LEFT JOIN [Dimension].[Payment Method] pm ON t.[Payment Method Key]=pm.[Payment Method Key]
GROUP BY GROUPING SETS ((tt.[Transaction Type],pm.[Payment Method]),(tt.[Transaction Type]),());
```

Actual output. Nine rows were returned. Detail included Customer Invoice/Unknown 198,043,439.45 and Supplier Invoice/EFT 1,085,993,513.06; the grand total was 2,988,689.65. Unlike CUBE, no unrelated year grouping was requested.

5.7.2 Top-N and Ranking Analysis

Ranking is useful in OLAP decision support because it orders aggregated entities and can identify leaders without discarding the underlying dimensions. The queries use SQL Server window functions and were executed against the actual facts.

```
WITH x AS (SELECT cu.[Customer],SUM(s.[Total Including Tax]) AS [Sales Including Tax]
FROM [Fact].[Sale] s JOIN [Dimension].[Customer] cu ON s.[Customer Key]=cu.[Customer Key]
GROUP BY cu.[Customer]), r AS
(SELECT [Customer],[Sales Including Tax],RANK() OVER (ORDER BY [Sales Including Tax] DESC) AS [Rank] FROM
x)
SELECT TOP 5 * FROM r ORDER BY [Rank],[Customer];
```

Actual output. The top five were Unknown 73,037,043.78 (rank 1), Tailspin Toys (Inguadona, MN) 438,689.81 (rank 2), Tailspin Toys (Minidoka, ID) 427,445.57 (rank 3), Wingtip Toys (Sarversville, PA) 420,001.08 (rank 4), and Tailspin Toys (Long Meadow, MD) 414,934.18 (rank 5).

```
WITH x AS (SELECT si.[Stock Item],d.[Calendar Year],SUM(s.[Profit]) AS [Profit]
FROM [Fact].[Sale] s JOIN [Dimension].[Stock Item] si ON s.[Stock Item Key]=si.[Stock Item Key]
JOIN [Dimension].[Date] d ON s.[Invoice Date Key]=d.[Date]
GROUP BY si.[Stock Item],d.[Calendar Year]), r AS
(SELECT [Calendar Year],[Stock Item],[Profit],ROW_NUMBER() OVER (PARTITION BY [Calendar Year] ORDER BY [Profit] DESC) AS [Rank] FROM x)
SELECT * FROM r WHERE [Rank]<=3 AND [Calendar Year]=2015 ORDER BY [Rank];
```

Actual output. The top three were 20 mm Double sided bubble wrap 50m, 1,782,960.00; Air cushion machine (Blue), 1,325,973.00; and 32 mm Anti static bubble wrap (Blue) 50m, 1,120,560.00.

```
WITH x AS (SELECT su.[Supplier],SUM(p.[Ordered Quantity]) AS [Ordered Quantity]
FROM [Fact].[Purchase] p JOIN [Dimension].[Supplier] su ON p.[Supplier Key]=su.[Supplier Key]
GROUP BY su.[Supplier]), r AS
(SELECT [Supplier],[Ordered Quantity],DENSE_RANK() OVER (ORDER BY [Ordered Quantity] DESC) AS [Rank] FROM
x)
SELECT TOP 5 * FROM r ORDER BY [Rank],[Supplier];
```

Actual output. Fabrikam, Inc. ranked first with 91,142,279, Litware, Inc. second with 48,626,764, followed by A Datum Corporation 5,076, The Phone Company 1,730, and Graphic Design Institute 1,442.

6 Comparison of OLAP Operations

Table 19: Comparison of OLAP operations

Operation	Main purpose	Dimensional effect	Typical SQL construct	Example used
CUBE	All grouping combinations	Subtotals in every combination	GROUP BY CUBE	Sales year/country
ROLLUP	Hierarchical subtotals	Detailed to parent to total	GROUP BY ROLLUP	Country/state/city
Drill-down	Higher to finer granularity	Adds lower hierarchy levels	Successive GROUP BY queries	Year/month/date
Slice	One fixed dimension value	Reduces one dimension	WHERE predicate	Year 2015
Dice	Multiple restrictions	Selects a subcube	Multiple WHERE predicates	Year/country/category
Pivot	Rotate an analytical view	Dimension values become columns	PIVOT	Profit by category/year
GROUPING SETS	Explicit aggregates	Only named combinations	GROUP BY GROUPING SETS	Country/year sets
Ranking	Comparative ordering	Orders aggregated members	RANK, ROW_NUMBER, DENSE_RANK	Top customers

7 Observations and Findings

The following observations are limited to values returned by the executed queries:

- Sales including tax was highest in the reported annual comparison in 2015 at 62,090,220.81 and lower in 2016 at 25,971,029.11.
- The sales data returned United States as the country member, with 8,950,628 sold units in the geographic drill-down.
- Novelty Shop generated 125,006,395.67 in sales including tax, compared with 73,037,043.78 for the N/A customer category.
- Fabrikam, Inc. was the largest supplier in the ranking query at 91,142,279 ordered units, ahead of Litware, Inc. at 48,626,764.
- The 2015 product ranking identified 20 mm Double sided bubble wrap 50m as the top displayed stock item by profit at 1,782,960.00.
- The transaction pivot separated positive invoice values from negative payment values and produced an overall net value of 2,988,689.65 in the executed grouping-set result.

8 Conclusion

WideWorldImportersDW is suitable for OLAP because its fact constellation shares conformed dimensions such as Date, Customer, Stock Item, Supplier, City, and Transaction Type. The executed experiments demonstrated CUBE, ROLLUP, drill-down, slice, dice, PIVOT, GROUPING SETS, and ranking using actual SQL Server data. Together, these constructs support multidimensional aggregation, controlled filtering, hierarchy navigation, presentation-oriented reshaping, and comparative decision support.

9 References

1. Microsoft, "WideWorldImportersDW database catalog," Microsoft Learn. [Microsoft Learn link](#).
2. Microsoft, "GROUP BY (Transact-SQL)," Microsoft Learn. [Microsoft Learn link](#).
3. Microsoft, "GROUPING (Transact-SQL)" and "GROUPING_ID (Transact-SQL)," Microsoft Learn. [GROUPING and GROUPING_ID](#).
4. Microsoft, "FROM - Using PIVOT and UNPIVOT," Microsoft Learn. [Microsoft Learn link](#).
5. Microsoft, "Ranking Functions (Transact-SQL)," Microsoft Learn. [Microsoft Learn link](#).
6. Microsoft, "ROW_NUMBER (Transact-SQL)," Microsoft Learn. [Microsoft Learn link](#).